

Module 1 — Core Java

Highest priority. Rounds 1 and 2 at every service company (TCS, Infosys, Cognizant, Accenture, Capgemini, Wipro, LTIMindtree) and the “Java fundamentals” screen at product companies live here. If you are shaky on JVM memory, OOP, and `String`, you will not pass.

Node.js bridge: You already know V8, the event loop, and `require`. Java’s JVM is V8’s much older, heavier cousin: explicit typing, ahead-of-time byte-code compilation, real OS threads (not a single event loop), and a garbage collector you can actually tune.

1.1 JVM, JDK, JRE & the Compilation Process

1. Why Interviewers Ask This

It’s the fastest way to tell a “Java developer” from someone who copied Spring code. If you can’t cleanly separate JDK/JRE/JVM you’ll be filtered in the first 5 minutes.

2. Core Concept

- **JDK** (Java Development Kit) = JRE + development tools (`javac`, `jar`, `javadoc`, `jdb`). You need it to *build*.
- **JRE** (Java Runtime Environment) = JVM + core libraries (`java.lang`, `java.util`...). You need it to *run*.
- **JVM** (Java Virtual Machine) = the abstract machine that executes byte-code. It’s a *specification*; HotSpot is Oracle’s implementation.

```
JDK  ⊇  JRE  ⊇  JVM
(build) (run) (execute bytecode)
```

3. Internal Working — Compilation & Execution

```
Person.java --javac--> Person.class (bytecode) --> JVM ClassLoader
                                     |
                                     +-----+-----+
                                     | Bytecode Verifier |
                                     +-----+-----+
                                     |
                               Interpreter (fast start) + JIT compiler
                                     |
                               native machine code (cached)
```

- `javac` compiles source to **platform-independent byte-code** (`.class`).
- JVM **interprets** byte-code initially (fast startup).
- The **JIT (Just-In-Time) compiler** watches for **hot** methods (default threshold ~10,000 invocations) and compiles them to native code. HotSpot has two JITs: **C1** (client, quick) and **C2** (server, aggressive optimizations); modern JVMs use **tiered compilation** (C1 first, then C2).
- “Write once, run anywhere” = byte-code is portable; only the JVM is platform-specific.

4. Memory Diagram

```
+-----+-----+ JVM Process -----+-----+
| ClassLoader Subsystem -> Runtime Data Areas -> Execution Engine |
|                           (Heap, Stacks,      (Interpreter,   |
|                           Metaspace, PC,      JIT, GC)         |
|                           Native Method Stack)                |
+-----+-----+-----+-----+
```

5. Real Production Example

On a Spring Boot service you ship a fat JAR. Build server needs the **JDK**; the container base image (`eclipse-temurin:21-jre`) only needs the **JRE** — smaller, fewer CVEs. JIT “warm-up” is why the *first* few requests after deploy are slow and p99 latency spikes right after a rolling restart.

6. Most Asked Interview Questions

- Difference between JDK, JRE, JVM? (*follow-up: which do you put in a Docker prod image?*)
- Is Java compiled or interpreted? (*both — bytecode compiled, then interpreted + JIT-compiled*)
- What is JIT? What is tiered compilation? (*follow-up: what is warm-up?*)
- Why is Java platform independent but the JVM is not?

7. Interview Traps

- Saying “Java is purely interpreted” (wrong — JIT compiles to native).
- Saying byte-code is machine code (it's not — it's JVM instructions).
- Thinking the JRE can compile code (it can't; no `javac`).

8. Best Answer

“`javac` compiles `.java` into portable byte-code. At runtime the JVM's class loader loads and verifies it, the interpreter runs it immediately for fast startup, and the JIT compiler promotes hot methods to native code using tiered C1/C2 compilation. The JDK is what I build with; production containers only ship a JRE to stay small and secure.”

9. Coding Example

```
// javac Hello.java -> Hello.class ; java Hello
public class Hello {
    public static void main(String[] args) {
        System.out.println("Bytecode ran on " + System.getProperty("java.version"));
    }
}
// Inspect bytecode: javap -c Hello
```

10. Follow-up Coding Questions

- Run `javap -c` on a class and explain a few opcodes (`iload`, `invokevirtual`).
- How would you force C2-only or disable tiered compilation? (`-XX:-TieredCompilation`)

11. Summary

JDK builds, JRE runs, JVM executes. Byte-code is portable; JIT makes it fast.

12. Cheat Sheet

Term	Contains	Purpose
JDK	JRE + javac/jar/javadoc	Development/build
JRE	JVM + libraries	Run apps
JVM	Class loader + memory + engine	Execute bytecode
JIT	C1 + C2 (tiered)	Bytecode → native

1.2 Class Loading

1. Why Interviewers Ask This

`ClassNotFoundException` vs `NoClassDefFoundError`, and how Spring/Tomcat isolate apps, all come from class loading. Frequent at product companies and any “we had a jar hell bug” story.

2. Core Concept

Classes are loaded lazily, on first active use, by a hierarchy of class loaders using **delegation**.

3. Internal Working — Loading → Linking → Initialization

```
1. Loading      : find .class bytes, create Class object in Metaspace
2. Linking
   a. Verification : bytecode is valid & safe
   b. Preparation  : static fields get default values (0/null/false)
   c. Resolution   : symbolic refs -> direct refs
3. Initialization : run static blocks & static initializers (assign real values)
```

Parent-delegation model: a loader asks its *parent* first; only if the parent can't find the class does the child try. Prevents core classes from being spoofed.

```
Bootstrap (rt/core, native) -> loads java.* (parent of all)
  ^
Platform/Extension loader   -> loads platform modules
  ^
Application/System loader    -> loads your classpath classes
  ^
(Custom loaders: web apps, plugins, OSGi)
```

4. Memory Diagram

```
new Foo() -> AppClassLoader.loadClass("Foo")
           -> ask Platform loader -> ask Bootstrap loader
           Bootstrap: "not mine" Platform: "not mine"
           App loader loads Foo.class -> Metaspace [Class<Foo>]
```

5. Real Production Example

Two web apps in one Tomcat each get their own `WebAppClassLoader`, so both can use different Jackson versions without clashing. `NoClassDefFoundError` in prod usually = the class was present at compile time but a dependency/jar is missing at runtime (classpath mismatch).

6. Most Asked Interview Questions

- Explain the class loading process. (3 phases)
- What is parent delegation and why? (security + no duplicates)
- `ClassNotFoundException` vs `NoClassDefFoundError`? (former: `Class.forName` can't find at runtime; latter: was there at compile, gone/failed init at runtime)
- When does a static block run? (at initialization, once, thread-safely)

7. Interview Traps

- Confusing the two class errors (very common).
- Thinking loading a class also creates instances (it doesn't).
- Saying static fields get real values in *preparation* (no — defaults there, real values in *initialization*).

8. Best Answer

“Loading finds the bytes and builds the **Class** object; linking verifies, gives static fields defaults, and resolves references; initialization runs static initializers once. Loaders delegate to their parent first so core **java.*** classes can't be overridden. That isolation is how one Tomcat runs two apps with conflicting library versions.”

9. Coding Example

```
public class Loaders {
    static { System.out.println("static init runs once, at initialization"); }
    public static void main(String[] a) {
        System.out.println(String.class.getClassLoader()); // null = Bootstrap
        System.out.println(Loaders.class.getClassLoader()); // AppClassLoader
    }
}
```

10. Follow-up Coding Questions

- Write a custom **ClassLoader** that loads a class from a byte array.
- Demonstrate lazy loading: prove a class isn't initialized until first active use.

11. Summary

Load → Link (verify, prepare, resolve) → Initialize. Parents get first refusal.

12. Cheat Sheet

Error	When	Cause
ClassNotFoundException	reflection/ Class.forName	name not on classpath
NoClassDefFoundError	linking/init	present at compile, missing/failed at runtime

1.3 Java Memory Model — Heap, Stack, Metaspace

1. Why Interviewers Ask This

Memory questions separate juniors from seniors and lead straight into GC and **OutOfMemoryError** debugging — a top production topic.

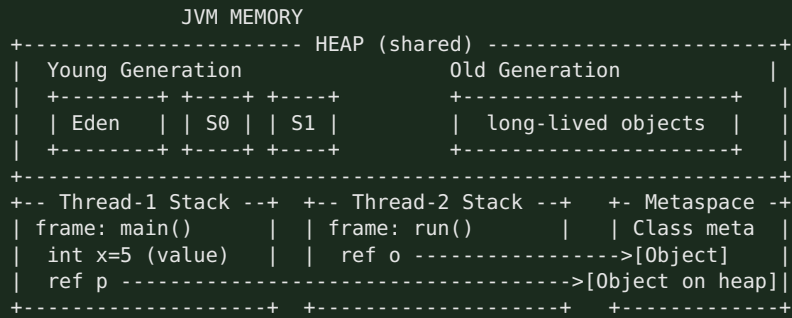
2. Core Concept

- **Heap** — shared, GC-managed; all objects & arrays live here.
- **Stack** — per-thread; holds frames with local variables and references (LIFO).
- **Metaspace** — off-heap (native memory since Java 8); class metadata, replaced PermGen.
- **PC register & Native method stack** — per thread, small.

3. Internal Working

- Each thread gets its own **stack**; each method call pushes a **frame** (locals, operand stack, return address). Frame pops on return. Deep recursion → **StackOverflowError**.
- **Heap** is split for generational GC: **Young** (Eden + two Survivor spaces) and **Old/Tenured**.
- **Metaspace** grows in native memory (bounded by **-XX:MaxMetaspaceOnly** if set); before Java 8 this was **PermGen** inside the heap and caused frequent **OutOfMemoryError: PermGen space**.
- **Primitives declared as locals** live on the stack; **object fields** (even primitive fields) live on the heap inside the object.

4. Memory Diagram



5. Real Production Example

A memory leak in a Spring app = objects unintentionally reachable (e.g., a static `Map` cache never evicted) → Old gen fills → `OutOfMemoryError: Java heap space`. You capture a heap dump (`-XX:+HeapDumpOnOutOfMemoryError`) and analyze in Eclipse MAT. `-Xmx2g -Xms2g` fixes heap size; classloader leaks (redeploys) show as Metaspace growth.

6. Most Asked Interview Questions

- Stack vs heap? Where do primitives/objects/references live? (follow-up: are *String literals* on stack? No — heap string pool)
- What replaced PermGen and why? (Metaspace, native memory, auto-grows)
- What causes `StackOverflowError` vs `OutOfMemoryError`?
- Is memory allocation thread-safe on the heap? (yes; TLABs make it fast)

7. Interview Traps

- “Objects can live on the stack” — no (ignoring escape-analysis scalar replacement, which is an optimization detail).
- “Primitives always on stack” — only *local* primitives; primitive *fields* are on the heap in the object.
- Confusing Metaspace (native) with heap.

8. Best Answer

“Objects live on the shared heap, split into young and old generations for GC. Each thread has its own stack of frames holding locals and references — references point into the heap. Class metadata moved from PermGen to native Metaspace in Java 8, which auto-grows so we stopped seeing PermGen OOMs. Heap OOM means a leak or undersized `-Xmx`; `StackOverflow` means unbounded recursion.”

9. Coding Example

```

public class MemoryDemo {
    int fieldOnHeap = 10;           // lives in the object on the heap
    void method() {
        int localOnStack = 5;      // stack
        MemoryDemo ref = new MemoryDemo(); // 'ref' on stack, object on heap
    }
    static int recurse(int n){ return recurse(n+1); } // -> StackOverflowError
}

```

10. Follow-up Coding Questions

- Trigger and catch a `StackOverflowError`; why can you catch an `Error` but shouldn't rely on it?
- Write code that causes a heap OOM with an ever-growing `List`.

11. Summary

Heap = objects (shared, GC). Stack = per-thread frames. Metaspace = class metadata (native).

12. Cheat Sheet

Area	Scope	Stores	Error
Heap	shared	objects/arrays	<code>OutOfMemoryError: Java heap space</code>
Stack	per-thread	frames, locals, refs	<code>StackOverflowError</code>
Metaspace	shared (native)	class metadata	<code>OutOfMemoryError: Metaspace</code>

1.4 Garbage Collection, Generational GC & Object Lifecycle

1. Why Interviewers Ask This

GC is *the* differentiator for backend Java. Everyone from TCS to Netflix asks “how does GC work” and “which collector do you use and why”.

2. Core Concept

GC automatically reclaims unreachable objects. **Reachability** (from GC roots), not reference counting, decides liveness. Generational GC exploits the **weak generational hypothesis**: *most objects die young*.

3. Internal Working — Generational GC

```
1. New objects -> Eden.
2. Eden full -> Minor GC: live objects copied to a Survivor space; dead ones dropped.
3. Survivors aged each Minor GC. After threshold (MaxTenuringThreshold) -> promoted to Old gen.
4. Old gen full -> Major/Full GC (slower, may 'stop the world').
```

- **GC Roots**: stack locals, static fields, active threads, JNI refs. Anything reachable from a root is live.
- **Mark-Sweep-Compact**: mark live, sweep dead, compact to remove fragmentation.
- **Collectors**:
- **Serial** — single thread, tiny heaps.
- **Parallel (Throughput)** — multi-thread, batch jobs.
- **G1 (default since Java 9)** — region-based, predictable pause targets (`-XX:MaxGCPauseMillis`).
- **ZGC / Shenandoah** — sub-millisecond pauses, huge heaps (low-latency services).
- **Stop-The-World (STW)**: application threads pause during certain GC phases. Minimizing STW is the whole game.

4. Memory Diagram

```
Allocation: [ Eden ] fills -> Minor GC -> survivors -> [ S0/S1 ] -> age -> [ Old ]
GC roots ---> reachable objects kept; unreachable islands (even if they reference
each other) are collected. Cyclic refs are NOT a leak in Java.
```

5. Real Production Example

A latency-sensitive payment service on a 8GB heap uses **G1** with `-XX:MaxGCPauseMillis=200`. You watch GC logs (`-Xlog:gc*`) for frequent Full GCs (a red flag for leaks or undersized old gen). Netflix-style low-latency services move to **ZGC** to keep pauses under 1ms.

6. Most Asked Interview Questions

- How does GC decide what to collect? (*reachability from GC roots, not ref counting*)
- Explain generational GC / why young & old? (*most objects die young → cheap minor GC*)
- Can you force GC? (`System.gc()` is only a hint; never rely on it)
- Difference between Minor, Major, Full GC? What is Stop-The-World?
- Which collectors do you know? When G1 vs ZGC? (*follow-up: does GC prevent all memory leaks? No.*)
- Do circular references cause leaks in Java? (*No — reachability handles cycles.*)

7. Interview Traps

- Saying Java uses reference counting (it doesn't — reachability).
- Claiming `System.gc()` guarantees collection.
- Saying "Java can't leak memory" — it can (unbounded caches, listeners, ThreadLocals, classloaders).
- Confusing `finalize()` with a destructor (see next section).

8. Best Answer

"The GC frees objects unreachable from GC roots — so cycles aren't a problem. It's generational because most objects die young: cheap minor GCs sweep Eden, survivors age and get promoted to old gen, which is collected less often by an expensive major GC. Modern JVMs default to G1, which is region-based and lets me target a max pause time; for ultra-low latency I'd use ZGC. `System.gc()` is only a hint, and GC doesn't fix logical leaks like an unbounded static cache."

9. Coding Example

```
public class GcDemo {
    public static void main(String[] args) {
        Object a = new Object();
        a = null;           // old object now unreachable -> eligible for GC
        System.gc();        // hint only; do NOT rely on it in code
        // Prefer WeakReference/soft caches to let GC reclaim under pressure:
        java.lang.ref.WeakReference<byte[]> cache =
            new java.lang.ref.WeakReference<>(new byte[1024]);
        System.out.println(cache.get() != null);
    }
}
```

10. Follow-up Coding Questions

- Write a class that leaks via a `static List` and explain how to detect it (heap dump/MAT).
- Explain `WeakReference` vs `SoftReference` vs `PhantomReference` with a cache example.

11. Summary

Reachability-based, generational, mostly automatic. G1 default; ZGC for low pause. GC ≠ leak-proof.

12. Cheat Sheet

Collector	Best for	Pause
Serial	tiny/CLI	high
Parallel	throughput/batch	medium
G1 (default)	general services	tunable (~200ms)
ZGC/Shenandoah	low latency, big heap	<1ms

Object lifecycle: Created (`new`) → In use (reachable) → Unreachable → Eligible → (optional `finalize`) → Collected.

1.5 OOP, SOLID & the Four Pillars

1. Why Interviewers Ask This

Every design and code-quality question rests on OOP + SOLID. "Explain SOLID with an example" is nearly guaranteed in service-company and mid-level product interviews.

2. Core Concept — Four Pillars

- **Encapsulation** — bundle state + behavior; hide internals behind methods (private fields, getters/setters, invariants).
- **Abstraction** — expose *what*, hide *how* (interfaces, abstract classes).
- **Inheritance** — reuse via “is-a” (`Dog extends Animal`).
- **Polymorphism** — one interface, many forms: **compile-time** (overloading) & **runtime** (overriding via dynamic dispatch).

3. Internal Working — Runtime Polymorphism

The JVM keeps a **vtable** (virtual method table) per class. An overridden call (`animal.sound()`) is resolved at runtime by the *actual object's* type via `invokevirtual`, not the reference type. Overloading is resolved at *compile time* by argument types via `invokestatic/invokevirtual` with a fixed signature.

4. Memory Diagram

```
Animal a = new Dog();           // ref type Animal, object type Dog
a.sound();
    -> invokevirtual -> look up Dog's vtable -> Dog.sound()    (runtime dispatch)
```

5. SOLID (with production framing)

Letter	Principle	One-liner	Spring example
S	Single Responsibility	one reason to change	Controller ≠ Service ≠ Repository
O	Open/Closed	open to extend, closed to modify	add a new <code>PaymentStrategy</code> impl, don't edit existing
L	Liskov Substitution	subtypes usable as base type	<code>Square extends Rectangle</code> violates it
I	Interface Segregation	many small interfaces > one fat	split <code>Repository</code> not one god-interface
D	Dependency Inversion	depend on abstractions	inject <code>PaymentGateway</code> interface, not <code>StripeClient</code>

6. Most Asked Interview Questions

- Explain the 4 OOP pillars with real examples.
- Explain SOLID; give a violation and the fix. (*follow-up: which SOLID does DI implement? → D*)
- Compile-time vs runtime polymorphism? (*overloading vs overriding*)
- Is Java 100% OOP? (*No — primitives exist.*)

7. Interview Traps

- Confusing abstraction with encapsulation (abstraction = design/hide complexity; encapsulation = data hiding + bundling).
- Saying overloading is runtime polymorphism (it's compile-time).
- Reciting SOLID with no example — interviewers want the *fix*.

8. Best Answer

“Encapsulation hides state behind methods to protect invariants; abstraction exposes intent through interfaces; inheritance reuses via is-a; polymorphism lets one reference behave as many types — overloading resolved at compile time, overriding at runtime via the vtable. SOLID keeps that flexible: e.g., Dependency Inversion is exactly what Spring DI gives us — my service depends on a `PaymentGateway` interface, and Spring injects Stripe or a mock.”

9. Coding Example

```
interface PaymentGateway { boolean charge(long cents); }           // abstraction + D + 0

final class StripeGateway implements PaymentGateway {              // encapsulated impl
    public boolean charge(long cents) { /* call Stripe */ return true; }
}

class CheckoutService {                                           // S: only checkout logic
    private final PaymentGateway gateway;                          // D: depends on abstraction
    CheckoutService(PaymentGateway gateway) { this.gateway = gateway; }
    boolean pay(long cents) { return gateway.charge(cents); }
}
```

10. Follow-up Coding Questions

- Refactor an `if/else` on payment type into the Strategy pattern (Open/Closed).
- Show a Liskov violation with `Rectangle/Square` and fix it.

11. Summary

4 pillars + SOLID = maintainable code. DI = Dependency Inversion in practice.

12. Cheat Sheet

Single · **O**pen-closed · **L**iskov · **I**nterface-seg · **D**ependency-inversion. Overload = compile-time; Override = runtime (vtable).

1.6 Interface vs Abstract Class · Overloading vs Overriding

Interface vs Abstract Class

	Interface	Abstract Class
Multiple inheritance	Yes (implement many)	No (extend one)
State/fields	only <code>public static final</code> constants	instance fields allowed
Methods	<code>abstract</code> + <code>default/static/private</code> (Java 8+)	<code>abstract</code> + <code>concrete</code>
Constructor	No	Yes
Use when	capability/contract ("can-do": <code>Comparable</code>)	shared base + state ("is-a")

Trap: Since Java 8, interfaces have `default` methods → the "interfaces can't have bodies" answer is outdated. The **diamond problem** with two default methods is resolved by forcing the class to override and call `Interface.super.method()`.

Overloading vs Overriding

	Overloading	Overriding
When	compile-time	runtime
Signature	must differ (params)	must match
Return type	can differ	same/covariant
Access	any	can't reduce visibility
<code>static</code>	can overload	can't override (hidden, not overridden)

Best answer for "can you override a static method?" → No. A static method belongs to the class; redeclaring it in a subclass **hides** it (resolved by reference type, not object type).

1.7 final · finally · finalize · static · this · super

final, finally, finalize (a guaranteed question)

- **final** — keyword. **final** variable = constant; **final** method = can't override; **final** class = can't extend (e.g., `String`).
- **finally** — block that always runs after **try/catch** (even on return/exception), used for cleanup. Skipped only on `System.exit()` or JVM crash.
- **finalize()** — Object method the GC *might* call before reclaiming. **Deprecated since Java 9, removed in later releases.** Unpredictable, can resurrect objects, hurts GC, no guarantee it runs. Use **try-with-resources** / `Cleaner` / `AutoCloseable` instead.

Best answer: “**final** is a modifier, **finally** is a cleanup block, **finalize()** is a deprecated GC hook I never use — I use try-with-resources for deterministic cleanup.”

static

Belongs to the **class**, not instances: shared across all objects, loaded at class initialization. Static methods can't use **this/super** or access instance members directly. Static blocks run once at init. Watch for the trap: static fields + concurrency = shared mutable state (needs synchronization).

this VS super

- **this** — current object (disambiguate fields, chain constructors `this(...)`).
- **super** — parent (call parent constructor `super(...)`, or parent method `super.foo()`).
- `super(...)/this(...)` must be the **first** statement in a constructor.

1.8 String Pool · String vs StringBuilder vs StringBuffer · Immutability

1. Why Interviewers Ask This

`String` is the most-used class and the classic immutability/`==` vs `.equals()` trap. Nearly guaranteed.

2 & 3. Core + Internal

- `String` is **immutable** and **final**. Its `char[]/byte[]` (compact strings since Java 9) is private and never mutated.
- **String Pool** (a.k.a. intern pool) lives in the **heap** (since Java 7; was PermGen before). String *literals* are interned — identical literals share one object. `new String("x")` creates a *new* heap object (not pooled unless you call `.intern()`).

4. Memory Diagram

```
String a = "hi";           // pool: ["hi"] <- a
String b = "hi";           // b -> same pooled "hi"
String c = new String("hi"); // heap: [new "hi"] <- c
                           //
String d = c.intern();      // d -> pooled "hi"
```

a == b	-> true
a == c	-> false
a.equals(c)	-> true
a == d	-> true

5. Real Production Example

Building a big log/CSV line by `s += token` in a loop creates a new `String` each iteration → $O(n^2)$ garbage. Use **StringBuilder** (single mutable buffer). In a multithreaded logger you'd use **StringBuffer** (synchronized) — or better, keep the builder thread-local.

Immutability — why `String` is immutable

Security (used in class loading, network, file paths), thread-safety (shareable without locks), safe as **HashMap keys** (cached `hashCode`), and pool sharing. Any “modification” returns a **new** `String`.

String vs `StringBuilder` vs `StringBuffer`

	String	StringBuilder	StringBuffer
Mutable	No	Yes	Yes
Thread-safe	Yes (immutable)	No	Yes (synchronized)
Speed	slow for concat loops	fastest	slower (locking)
Use	fixed text, keys	single-thread build	rare, shared build

6. Most Asked Interview Questions

- Why is `String` immutable? (*security, thread-safety, hashcode caching, pool*)
- `==` vs `.equals()` for strings? (*reference vs value*)
- `new String("a") == "a"`? (*false; `.equals` true; `.intern()` fixes*)
- `String` vs `StringBuilder` vs `StringBuffer`? When each?
- How many objects does `String s = new String("a")` create? (*up to 2: literal in pool + heap object*)

7. Interview Traps

- Using `==` to compare string content.
- Saying the pool is in PermGen (it moved to heap in Java 7).
- Using `String +=` in loops.

8. Best Answer

“`String` is immutable and `final` for security, thread-safety, and so it can be pooled and safely cached as a map key. Literals are interned so equal literals share one object, which is why `==` can be true for literals but false against `new String`. For heavy concatenation I use `StringBuilder`; `StringBuffer` only when a buffer is genuinely shared across threads.”

9. Coding Example

```
String a = "hi", b = "hi";
System.out.println(a == b);           // true (pool)
System.out.println(a == new String("hi")); // false (new heap object)
System.out.println(a.equals(new String("hi"))); // true

StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) sb.append(i).append(','); // O(n), no garbage storm
String csv = sb.toString();
```

10. Follow-up Coding Questions

- Reverse a string in place using `StringBuilder.reverse()`; then without it.
- Explain why using a mutable object as a `HashMap` key is dangerous.

11 & 12. Summary + Cheat Sheet

Immutable `String` → safe & poolable. `==` = reference, `.equals()` = value. Build with `StringBuilder`; synchronize with `StringBuffer` only if shared.

1.9 Immutable Objects · Wrapper Classes · Autoboxing

Building an immutable class (common coding ask)

```
public final class Money {
    // 1. final class (no subclass)
    private final long cents;
    private final String currency;
    // 2. private final fields
    public Money(long cents, String currency) { // 3. set via constructor
        this.cents = cents;
        this.currency = currency; // 4. defensively copy mutable inputs (none here)
    }
    public long cents() { return cents; } // 5. getters only, no setters
    public String currency() { return currency; }
}
```

Rules: `final` class, all fields `private final`, no setters, defensive copies of mutable fields (arrays, `Date`, collections) on the way in and out.

Wrapper Classes & Autoboxing

- Wrappers box primitives into objects: `int`→`Integer`, `long`→`Long`, etc. Needed for generics/collections (`List<Integer>`).
- **Autoboxing** = automatic `int` → `Integer`; **unboxing** = reverse. Compiler inserts `Integer.valueOf/intValue`.
- **Integer cache**: `valueOf` caches -128..127, so `Integer a=100, b=100; a==b` is `true`, but `127 ... Integer a=200, b=200; a==b` is `false` (different objects). Always compare wrappers with `.equals()`.
- **NPE trap**: unboxing a `null Integer` into an `int` throws `NullPointerException`.

Traps: `Integer ==` comparisons; NPE on unboxing null; autoboxing in tight loops (creates garbage — prefer primitives for performance-critical code).

1.10 Enum · var · Records · Sealed Classes

Enum

Type-safe set of constants; each is a singleton. Can have fields, constructors, methods, and per-constant bodies. Great for strategy/state. Use in `switch`; use `EnumMap/EnumSet` (very fast). `enum` is implicitly `final` and extends `java.lang.Enum`.

```
enum Status {
    ACTIVE(1), INACTIVE(0);
    private final int code;
    Status(int code){ this.code = code; }
    int code(){ return code; }
}
```

var (Java 10) — local variable type inference

Only for **local** variables with an initializer. Not for fields, params, or return types. Still statically typed — `var list = new ArrayList<String>();` infers `ArrayList<String>`. Don't overuse; keep readability.

Records (Java 16) — basic awareness

Immutable data carriers: auto-generate constructor, `equals`, `hashCode`, `toString`, accessors.

```
public record Point(int x, int y) {} // that's a full immutable value class
```

Great for DTOs. Can't extend classes; are implicitly `final`. Interviewers love "records vs Lombok `@Value`".

Sealed Classes (Java 17) — basic awareness

Restrict which classes may extend/implement: `sealed interface Shape permits Circle, Square {}`. Enables exhaustive `switch` and controlled hierarchies. Just know the concept and syntax.

Module 1 — Top 25 Interview Questions (with senior answers)

1. **JDK vs JRE vs JVM?** JDK builds (has `javac`), JRE runs (JVM + libs), JVM executes byte-code. Ship a JRE in prod.
2. **Is Java compiled or interpreted?** Both — compiled to byte-code, then interpreted + JIT-compiled to native.
3. **What is JIT / tiered compilation?** Promotes hot methods to native via C1 then C2.
4. **Class loading phases?** Loading → Linking (verify, prepare, resolve) → Initialization.
5. **Parent delegation?** Child asks parent first — prevents core-class spoofing & duplicates.
6. **ClassNotFoundException vs NoClassDefFoundError?** Reflection-time missing vs runtime-missing after compile.
7. **Stack vs Heap?** Per-thread frames/locals vs shared objects (GC-managed).
8. **What replaced PermGen?** Metaspace (native memory, auto-grows) in Java 8.
9. **How does GC decide liveness?** Reachability from GC roots, not reference counting.
10. **Generational GC?** Most objects die young → cheap minor GC of Eden; survivors promoted to old gen.
11. **Default collector? Low-latency choice?** G1 default; ZGC/Shenandoah for sub-ms pauses.
12. **Can `System.gc()` force GC?** No — only a hint.
13. **Do cycles leak in Java?** No — reachability handles cycles. Leaks come from unbounded reachable structures.
14. **final vs finally vs finalize?** Modifier vs cleanup block vs deprecated GC hook.
15. **Why is finalize deprecated?** Unpredictable, no guarantee, hurts GC — use try-with-resources/Cleaner.
16. **4 OOP pillars?** Encapsulation, Abstraction, Inheritance, Polymorphism.
17. **Explain SOLID + a fix.** e.g., Dependency Inversion → inject an interface, not a concrete client.
18. **Overloading vs overriding?** Compile-time (params) vs runtime (vtable dispatch).
19. **Can you override static/private/final?** No to all — static is hidden; private/final aren't inherited/overridable.
20. **Interface vs abstract class?** Multiple inheritance + contract vs single inheritance + shared state.
21. **Why is String immutable?** Security, thread-safety, hashCode caching, pooling.
22. **== vs .equals()** ? Reference identity vs logical equality.
23. **String vs StringBuilder vs StringBuffer?** Immutable vs mutable-unsynchronized vs mutable-synchronized.
24. **Integer caching / autoboxing traps?** -128..127 cached; compare wrappers with `.equals()`; null unboxing NPE.
25. **What are records/sealed classes for?** Immutable data carriers; restricted hierarchies for exhaustive matching.

Module 1 — Top Coding Questions

- Build an immutable `Money/Employee` class (with defensive copies).
- Implement `equals()` and `hashCode()` correctly (and explain the contract).
- Reverse a string; check palindrome; count char frequency with a `Map`.
- Demonstrate the `Integer` cache boundary (127 vs 128).
- Write a class hierarchy showing runtime polymorphism, then refactor an `if/else` type-switch into Strategy.

Module 1 — Common Follow-ups

- “Show me the `equals/hashCode` contract.” (equal objects → equal hash; used by `HashMap`.)
- “How would you debug an OOM in prod?” (heap dump + MAT, GC logs, check caches/`ThreadLocals`.)
- “Why not just use Lombok instead of records?” (records are language-level, immutable, no build-time processor.)

Module 1 — One-Page Cheat Sheet

```
JDK⊃JRE⊃JVM | javac->bytecode->JIT(C1/C2 tiered)
ClassLoader: Load->Verify/Prepare/Resolve->Init | parent-delegation
Heap=objects(GC) Stack=frames(per-thread) Metaspace=class meta(native)
GC: reachability from roots; generational (Eden->Survivor->Old); G1 default, ZGC low-latency
System.gc()=hint. Cycles don't leak. finalize=deprecated -> try-with-resources
OOP: Encapsulation/Abstraction/Inheritance/Polymorphism
SOLID: S O L I D; DI = Dependency Inversion
Overload=compile-time(params); Override=runtime(vtable). static/private/final NOT overridable
Interface: multi-inherit + default methods; Abstract: single + state
String immutable+pooled; ==ref vs .equals value; StringBuilder(fast) StringBuffer(sync)
Integer cache -128..127; compare wrappers with equals; null unbox=NPE
record=immutable DTO; sealed=restricted hierarchy; var=local inference
```

Module 1 — Mock Interview (answer these, then continue)

1. “Walk me through what happens from `javac Foo.java` to the method running as native code.”
2. “My Spring service throws `OutOfMemoryError: Java heap space` at 3 a.m. under load. Walk me through diagnosis and fixes.”
3. “Why can two web apps in one Tomcat use different Jackson versions?”
4. “Give me a real SOLID violation you fixed and how DI helped.”
5. “`Integer a = 128, b = 128; a == b` — true or false, and why? What about `127`?”
6. “Write an immutable `Money` class and justify every keyword.”

Model answers are embedded in the sections above. When ready, continue to Module 2.